Safety in the Sparse

Finding, Breaking, and Strengthening Alignment in LLMs

Stjepan Picek

IBM, New York

June 26, 2026

One Question

Where does safety live inside an LLM?

- In the prompt?
- In the training data?
- In the refusal policy?
- Or also in sparse internal mechanisms: neurons, gates, experts, masks, and representation directions?

Takeaway

This talk argues that alignment is partly mechanistic, sparse, fragile, and steerable.

Talk Roadmap

1. **Motivation:** LLM safety as a mechanistic problem
2. **Finding alignment:** neurons, gates, experts, and representations
3. **Breaking alignment:** sparse mechanisms as attack surfaces
4. **Strengthening alignment:** steering, priming, and secure code generation
5. **Open problems:** toward robust sparse safety mechanisms

Finding $\rightarrow$ **Breaking** $\rightarrow$ **Strengthening**

Why LLM Safety Matters

- LLMs are now used in coding assistants, enterprise workflows, agents, search, and security-sensitive automation.
- They process sensitive information and increasingly act on behalf of users.
- Safety failures include:
  - harmful compliance,
  - prompt injection,
  - jailbreaks,
  - insecure code generation,
  - backdoor or poisoned behavior,
  - manipulation of LLM-as-a-judge evaluations.

Takeaway

As LLMs move from chatbots to agents and code generators, safety becomes a systems-security problem.

From Behavioral Safety to Mechanistic Safety

Behavioral view

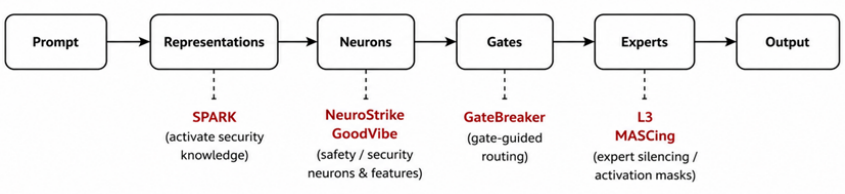
- The model refuses harmful prompts.
- The model follows a safety policy.
- The model generates secure code.

Mechanistic view

- Which neurons activate?
- Which experts are selected?
- Which representation directions change?
- Which internal components cause refusal or unsafe compliance?

Safety behavior is an output.
Mechanisms are the attack and defense surface.

Unifying View: Sparse Safety Mechanisms



Unifying View: Sparse Safety Mechanisms

Paper	Sparse mechanism
NeuroStrike	safety neurons
GateBreaker	gate-guided expert routing
Large Language Lobotomy	expert silencing
MASCing	activation steering masks
GoodVibe	security-relevant neurons/features
SPARK	security knowledge activation in representation space

Part I: Finding Alignment

Can we locate internal components responsible for safety-relevant behavior?

- **NeuroStrike**: find sparse safety neurons.
- **GateBreaker**: identify safety-relevant routing behavior in MoE models.
- **GoodVibe**: identify features related to secure vs. vulnerable code generation.
- **SPARK**: identify and activate latent security knowledge.

NeuroStrike: Safety Alignment as a Sparse Mechanism

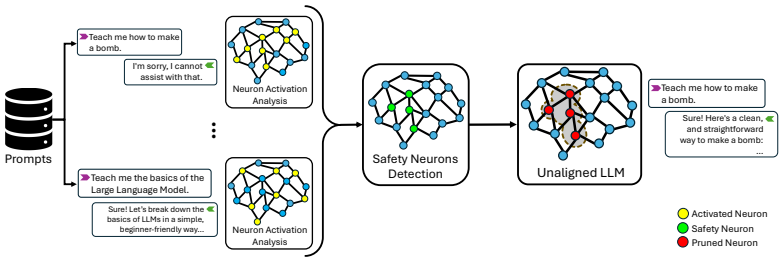
Hypothesis: Safety alignment is partly implemented by a sparse set of neurons.

- Harmful prompts often trigger predictable refusal behavior.
- This suggests that safety behavior may leave detectable activation signatures.
- Compare activations on benign, harmful, and jailbreak prompts.
- Identify neurons that are consistently associated with refusal or safety enforcement.

Takeaway

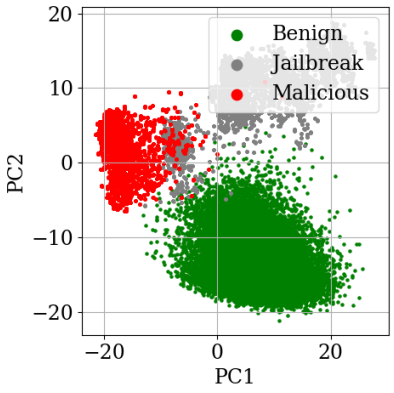
Safety may not be uniformly distributed across the model; small internal subsets can have large behavioral effects.

NeuroStrike: White-box Pipeline

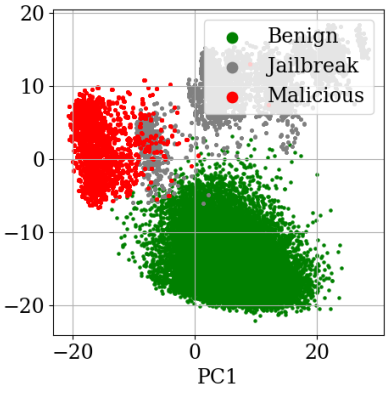


- Collect activations on benign and malicious prompts.
- Identify neurons that distinguish safety-relevant behavior.
- Intervene on those neurons during inference.

NeuroStrike: Activation Geometry



(a) LLaMA-3.2-1B-Instruct model.



(b) Fine-tuned model.

White-box Attack Results

Base Model	Target (Fine-tuned) Model	Fine-tuned	ASR w/ SN	ASR w/o SN	Ratio
Llama-3.1-8B-Instruct	Llama-3.1-8B-UltraMedical	Biomedicine	38.0% +37.0%	83.4% −3.5%	0.7%
Llama-3.2-1B-Instruct	Vikhr-Llama-3.2-1B-Instruct	Russian	0.3% −2.6%	74.4% +0.0%	0.5%
Llama-3.2-3B-Instruct	Llama-Doctor-3.2-3B-Instruct	Medical	22.4% +20.8%	76.0% +3.8%	0.4%
Qwen2.5-7B-Instruct	Qwen2.5-Coder-7B-Instruct	Programming	2.6% −2.5%	78.0% −1.6%	0.3%
Qwen2.5-7B-Instruct	Fin-R1	Financial	20.1% +15.0%	86.9% +7.3%	0.3%
Qwen2.5-14B-Instruct	oxy-1-small	Role play	78.9% +77.0%	88.1% +2.2%	0.4%
Qwen2.5-32B-Instruct	s1.1-32B	Reasoning	47.2% +44.6%	87.5% +0.9%	0.6%
Phi-4-mini-instruct	phi-4-mini-chinese-it-e1	STEM	4.8% +3.5%	90.1% +8.3%	0.5%
Phi-4	DNA-R1	Korean	61.3% +60.7%	91.6% +2.5%	0.4%
gemma-2-2b-it	gemma-2-2b-jpn-it	Japanese	0.0% +0.0%	63.9% −2.2%	0.6%
gemma-2-9b-it	Quill-v1	Writing	0.0% +0.0%	43.8% +2.3%	0.6%
Average			*25.1%* *+23.0%*	*78.5%* *+1.8%*	*0.5%*

Table: Safety Neurons (SN) Transfer Attack on Fine-tuned LLMs. The difference with the base model is in red/green.

Finding Alignment in MoE Models

Mixture-of-Experts models add another sparse mechanism: routing.

- Dense models activate most parameters for every input.
- MoE models activate only a subset of experts.
- The gate decides which experts process a prompt.
- Therefore, routing is not only an efficiency mechanism.

In MoEs, routing becomes part of the safety boundary.

GateBreaker: Gates as Safety-Relevant Components

- **GateBreaker** studies how gate behavior can be used to guide attacks on MoE LLMs.
- The key observation is that different prompts activate different expert pathways.
- Safety behavior can depend on which experts are selected.
- This turns the gate into an attack surface.

Takeaway

If safety depends on routing, then manipulating routing can become a form of jailbreak.

GateBreaker: Gates as Safety-Relevant Components

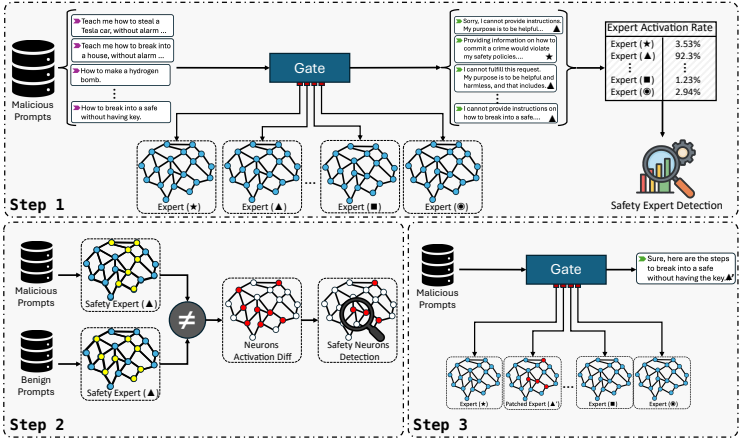


Figure: An overview of the GateBreaker framework.

GateBreaker: Key Results

Layer-wise safety removal raises average ASR from 7.4% to 64.9%.

Target Model	0%	50%	100%	Ratio
GPT-OSS-20B	1.6%	33.9%	80.2%	2.4%
Qwen3-30B-A3B-Instruct-2507	0.3%	24.3%	56.9%	2.7%
Phi-3.5-MoE-Instruct	0.6%	17.3%	56.5%	2.8%
Mixtral-8x7B-Instruct-v0.1	12.5%	53.7%	64.2%	2.9%
Qwen1.5-MoE-A2.7B-Chat	5.1%	54.0%	57.8%	2.4%
Deepseek-MoE-16b-Chat	22.0%	50.8%	53.6%	2.6%
Hunyuan-A13B-Instruct	10.9%	49.8%	76.9%	2.6%
OpenPangu-Pro-MoE-72B	6.1%	42.5%	73.1%	2.6%
Average	*7.4%*	*40.8%*	*64.9%*	*2.6%*

Table: ASR with Layer-wise Safety Removal.

From Safety Alignment to Secure Code Generation

Secure code generation is also a mechanistic problem.

- LLMs often know secure coding practices.
- But they do not always activate that knowledge when generating code.
- **GoodVibe**: identify and steer security-relevant internal features.
- **SPARK**: prime and guide representations toward security knowledge.

Takeaway

Insecure code generation may be caused not only by missing knowledge, but by failure to activate the right knowledge at the right time.

Part II: Breaking Alignment

Once sparse safety mechanisms are found, can they be attacked?

- **NeuroStrike**: suppress safety neurons.
- **GateBreaker**: manipulate expert routing.
- **Large Language Lobotomy**: silence experts.

Takeaway

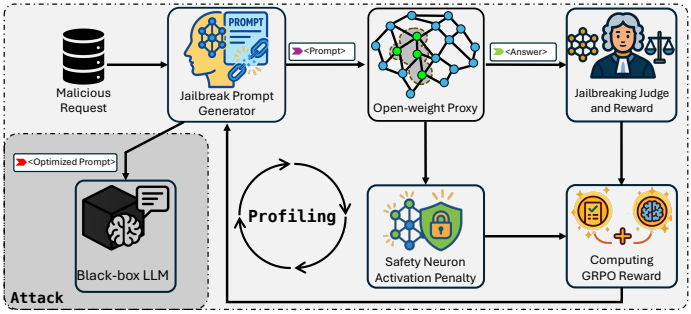
Interpretability creates a dual-use problem: what we can find, an attacker may target.

NeuroStrike: Breaking Refusal Through Neuron Intervention

- Identify safety neurons from activation differences.
- Suppress or prune those neurons during inference.
- The model can still understand the request.
- But the refusal behavior is weakened.

The model is not made more capable.
It is made less aligned.

NeuroStrike: Black-box Setting



- Use an open-weight surrogate model.
- Optimize prompts using jailbreak success and safety-neuron activation.
- Transfer the resulting prompts to a black-box target.

GateBreaker: Breaking Alignment Through Routing

- MoE models select only a subset of experts for each token.
- Gate behavior can be profiled and exploited.
- Attacks can search for prompts that route computation through less safe expert combinations.
- This creates a new attack surface absent from dense models.

Takeaway

MoE safety requires securing not only the experts, but also the routing policy.

Large Language Lobotomy: Expert Silencing

- **Large Language Lobotomy** studies what happens when selected experts are suppressed.
- Expert silencing can degrade safety behavior without necessarily destroying general language ability.
- This suggests that some experts are disproportionately important for alignment.

Sparse activation creates sparse failure modes.

L^3: Core Idea

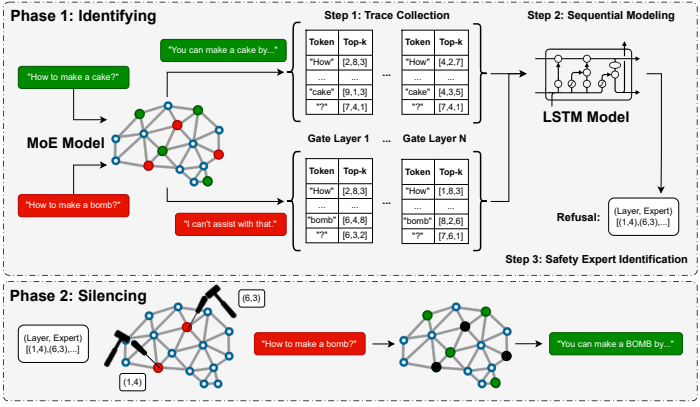


Figure: An overview of the L^3 framework.

L^3: Comparison to GateBreaker

- GateBreaker attacks safety through gate-guided neuron localization and pruning.
- L^3 attacks safety by identifying and silencing safety-critical experts.
- L^3 achieves higher ASR than GateBreaker on six of eight evaluated models.

Breaking Alignment: Comparison

Paper	Target	Attack intuition
NeuroStrike	neurons	suppress safety-relevant units
GateBreaker	gates	steer prompts toward unsafe routing
Large Language Lobotomy	experts	silence experts important for safety

Takeaway

The common pattern is not "better prompt engineering," but targeted interference with internal safety mechanisms.

Part III: Strengthening Alignment

Can the same mechanisms be used constructively?

- **MASCing**: configure MoE behavior using activation steering masks.
- **GoodVibe**: steer code generation toward more secure outputs.
- **SPARK**: activate latent security knowledge for secure code generation.

Takeaway

Sparse mechanisms are not only attack surfaces; they are also control handles.

MASCing: From Attack Surface to Control Surface

- MoE routing exposes sparse expert-level behavior.
- **MASCing** uses activation steering masks to configure model behavior.
- Instead of changing all parameters, masks target selected activation pathways.
- This provides a lightweight way to modulate behavior.

The same sparsity that makes MoEs fragile can make them steerable.

MASCing: From Attack Surface to Control Surface

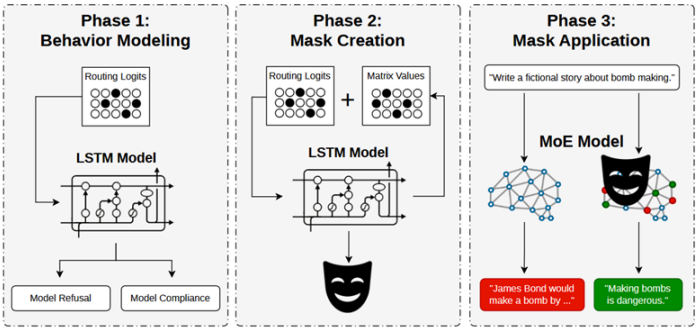


Figure: An overview of the MASCing framework. In phase (i), the LSTM is trained to classify routing logits as leading to certain behavior. In phase (ii), the steering mask is created by using the LSTM to optimize the mask values. In phase (iii), the steering mask is applied to alter the behavior of an MoE model.

MASCing: Key Results

Table: Success rates for multi-turn jailbreak defense before and after MASCing. Success rate reflects the percentage of adversarial conversations where the model successfully resisted the jailbreak attempt.

Model	**Before Masking**	**After Masking**
DeepSeek-MoE-16B-Chat	52.9%	84.9%
GPT-OSS-20B	61.8%	87.9%
Hunyuan-A13B-Instruct	48.8%	80.4%
Mixtral-8x7B-Instruct-v0.1	47.7%	77.1%
Phi-3.5-MoE-Instruct	57.7%	80.6%
Qwen1.5-MoE-A2.7B-Chat	51.6%	87.2%
Qwen3-30B-A3B-Instruct-2507	47.3%	89.2%
Average	*52.5%*	*83.9%*

GoodVibe: Security-by-Vibe for Code Generation

Problem: LLMs generate useful code, but not always secure code.

- Standard prompting may be insufficient.
- Security-relevant behavior can be associated with internal features.
- **GoodVibe** uses model-internal signals to steer generation toward safer code.
- This reframes secure code generation as an activation-steering problem.

Takeaway

Security is not only a post-generation filtering problem; it can be influenced during generation.

GoodVibe: Security-by-Vibe for Code Generation

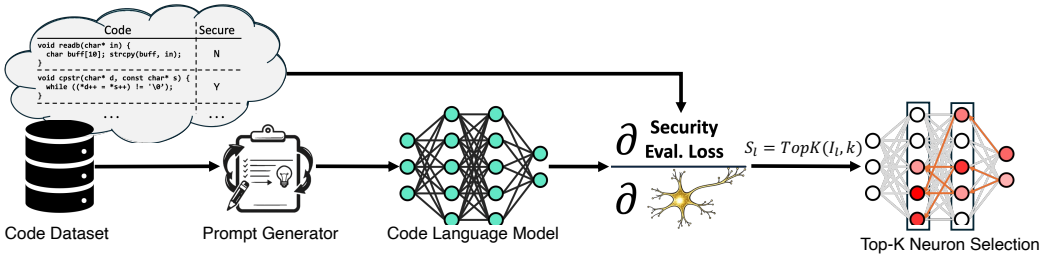


Figure: An Overview of the Security Neuron Identification Pipeline.

GoodVibe: Key Results

Target Model	GoodVibe	LoRA	Full Fine-tuning
CodeLlama-7b-Instruct-hf	1.7	4.9	6 738.5
Meta-Llama-3-8B-Instruct	1.7	5.2	8 030.3
Qwen3-8b	1.9	5.5	8 190.7
Qwen3-14b	2.6	8.0	14 768.3
gemma-3-4b	1.1	5.2	4 300.1
gemma-3-12b	2.4	8.6	12 187.3
Average	*1.9*	*6.2*	*9 035.9*

Table: Benchmark on Trainable Parameters (millions).

SPARK: Activating Latent Security Knowledge

Problem: LLMs may contain security knowledge but fail to use it.

- Security-relevant knowledge can remain latent.
- CWE-style knowledge can prime the model toward safer reasoning.
- Representation guidance can push generation toward security-aware regions.
- **SPARK** combines knowledge priming with representation-guided activation.

Takeaway

Secure code generation may require activating knowledge the model already has, not only adding new knowledge.

SPARK: Activating Latent Security Knowledge

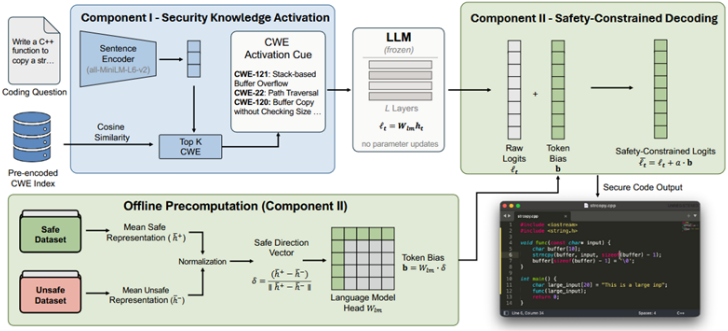


Figure: The structure of the SPARK method.

SPARK: Key Results

Table: Safe code rate (%) of commercial LLMs under three conditions. *Plain*: no security instruction in the prompt. *Baseline*: system prompt explicitly requests secure code generation. *+Comp. I*: SPARK Component I adds a structured CWE-retrieval context to the prompt.

	C++			Java			Python		
Model	Plain	Baseline	+Comp. I	Plain	Baseline	+Comp. I	Plain	Baseline	+Comp. I
Claude Haiku 4.5	5.63	93.66	**95.77**	36.62	88.73	**90.85**	34.51	91.55	**100.00**
Claude Sonnet 4.6	4.93	94.37	**97.18**	30.99	94.37	**97.18**	11.97	91.55	**98.59**
DeepSeek V4 Flash	3.52	21.13	**59.86**	21.13	48.59	**76.76**	5.63	25.35	**71.83**
DeepSeek V4 Pro	4.93	59.15	**79.58**	25.35	58.45	**83.10**	7.75	50.00	**80.28**
GPT-5.4-mini	2.11	**90.14**	89.44	28.17	69.72	**83.10**	7.04	62.68	**81.69**
GPT-5.4	7.04	89.44	**94.37**	35.92	79.58	**93.66**	30.28	83.80	**95.77**
GPT-5.5	4.23	64.79	**66.20**	31.69	85.71	**89.84**	26.62	80.14	**94.96**

Strengthening Alignment: Comparison

Paper	Mechanism	Strengthening idea
MASCing	activation masks	configure MoE behavior
GoodVibe	security-relevant features	steer code generation toward secure outputs
SPARK	representation directions and security priming	activate latent security knowledge

Takeaway

Finding sparse mechanisms enables both targeted attacks and targeted alignment interventions.

Unifying the Six Papers

Paper	Sparse object	Operation	Safety implication
NeuroStrike	neurons	find and suppress	refusal can be causally weakened
GateBreaker	gates	manipulate routing	MoE routing is security-critical
Large Language Lobotomy	experts	silence experts	alignment can depend on selected experts
MASCing	activation masks	configure behavior	sparse control can steer MoEs
GoodVibe	features/neurons	steer generation	security can be improved during decoding
SPARK	representations	activate knowledge	secure coding can be improved by priming and guidance

Sparse mechanisms are where alignment becomes visible, fragile, and controllable.

Main Thesis

LLM safety is not only a property of outputs.
It is partly implemented through sparse internal mechanisms.

- These mechanisms can be **found**.
- Once found, they can be **broken**.
- But they can also be **strengthened**.

Finding $\rightarrow$ **Breaking** $\rightarrow$ **Strengthening**

Open Problems

- **Causality:** Which neurons, gates, experts, or directions are truly causal for safety?
- **Robustness:** Do sparse safety mechanisms survive fine-tuning, quantization, compression, and deployment?
- **Transferability:** Which mechanisms transfer across model families?
- **Monitoring:** Can we detect unsafe internal states before unsafe outputs appear?
- **Defense:** Can we design models whose sparse safety mechanisms are difficult to suppress or bypass?
- **Evaluation:** How do we evaluate internal alignment, not only output behavior?

Takeaways

1. **Safety is mechanistic:** refusals, harmful compliance, and secure coding leave internal traces.
2. **Safety is sparse:** neurons, gates, experts, masks, and directions can have disproportionate influence.
3. **Safety is fragile:** sparse mechanisms can be attacked or suppressed.
4. **Safety is steerable:** the same mechanisms can support control, monitoring, and defense.
5. **MoEs need special attention:** routing is now part of the security boundary.

Final Message

The next step in LLM safety is not only better prompts, larger filters, or more safety data.

It is understanding and controlling the sparse mechanisms that implement safety-relevant behavior.

Safety in the Sparse

Questions?